JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface DoubleConsumer

All Known Subinterfaces:

`DoubleStream.Builder`

All Known Implementing Classes:

`DoubleSummaryStatistics`

Functional Interface:

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

```
@FunctionalInterface
public interface DoubleConsumer
```

Represents an operation that accepts a single `double`-valued argument and returns no result. This is the primitive type specialization of `Consumer` for `double`. Unlike most other functional interfaces, `DoubleConsumer` is expected to operate via side-effects.

This is a functional interface whose functional method is `accept(double)`.

Since:

1.8

See Also:

`Consumer`

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
void	**accept**(double value)	Performs this operation on the given argument.
default DoubleConsumer	**andThen** (DoubleConsumer after)	Returns a composed DoubleConsumer that performs, in sequence, this operation followed by the after operation.

Method Details

accept

void accept(double value)

Performs this operation on the given argument.

Parameters:

value - the input argument

andThen

default DoubleConsumer andThen(DoubleConsumer after)

Returns a composed DoubleConsumer that performs, in sequence, this operation followed by the after operation. If performing either operation throws an exception, it is relayed to the caller of the composed operation. If performing this operation throws an exception, the after operation will not be performed.

Parameters:

after - the operation to perform after this operation

Returns:

a composed DoubleConsumer that performs in sequence this operation followed by the after operation

Throws:

NullPointerException - if after is null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.